

Vialink Backend

Guía rápida para el Frontend — qué tienes listo, cómo conectarte, qué eventos llegan

Para: Sebastián (Frontend Lead)

Este documento te explica **en lenguaje claro** qué tiene listo el backend, cómo conectarte, y qué información te llega en tiempo real. Para el contrato técnico completo (todos los campos, tipos, etc.) ve a docs/api-contract.md en el repo del backend. Este PDF es el resumen ejecutivo.

Estado actual

El backend está **desplegable y funcional**. Hay 86 buses moviéndose en tiempo real en el mapa de Barranquilla, el asistente de IA responde en español natural usando datos reales, y todo lo que pasa (viajes, incidentes, alertas) se emite por WebSocket para que tu UI reaccione al instante.

Hay 29 endpoints REST, 1 gateway WebSocket con 6 tipos de eventos, y 3 procesos corriendo en segundo plano que mantienen los datos vivos sin que nadie los dispare.

1 · Qué es Vialink

Webapp de transporte público inteligente para Barranquilla. El usuario abre el mapa y ve qué buses vienen al lugar exacto donde está parado — sin necesidad de un paradero fijo, porque en Colombia los buses tradicionales paran a demanda en cualquier punto de su recorrido.

El diferenciador #1 es el **asistente de IA conversacional** (Claude Haiku 4.5) que entiende preguntas en español natural como “¿Cómo llego al Centro si voy de afán?” y responde con la mejor ruta y tiempo real de llegada, no con datos mock.

Lo que diferencia a Vialink de otras apps:

- Cubre los **buses tradicionales** (70% del transporte real de Colombia), no solo los masivos como Transmetro.
- El usuario hace tap en **cualquier punto** del mapa, no en un paradero fijo, y obtiene los próximos buses.
- El asistente responde con **datos reales del backend**, no respuestas pre-cocinadas.
- Vista admin del pitch mostrando 500 usuarios virtuales (*en construcción*) usando la app en vivo.

2 · Qué tienes a disposición HOY

Para el mapa principal

Pregunta	Qué te da el backend
¿Qué lugares populares hay cerca?	Lista de los 80 puntos famosos de BAQ (universidades, malls, hospitales, plazas, barrios) ordenados por distancia, con cuántas rutas pasan cerca de cada uno.
¿Qué rutas pasan por aquí?	Lista ligera de las rutas cuyo recorrido pasa cerca del punto que indiques (radio configurable).
¿Cuándo viene el próximo bus a este punto exacto?	Endpoint estrella. Para cada ruta cercana: próximos 3 buses con su ETA en segundos y distancia en metros. Con caché de 3 segundos para que el mapa pueda llamarlo agresivamente.
¿Cómo dibujo el recorrido de una ruta?	GeoJSON LineString listo para <code>L.geoJSON()</code> de Leaflet o <code>map.addSource()</code> de Mapbox.

¿Qué buses están activos en una ruta?

Snapshot de todos los buses de esa ruta con su posición, velocidad, dirección. **Úsalo solo para inicializar el mapa;** después actualízalos con el WebSocket.

Para el detalle de un lugar / búsqueda

Pregunta	Qué te da el backend
Buscar un lugar por nombre	Búsqueda fuzzy : tolera typos y falta de tildes. <i>“olimpica”</i> encuentra <i>Olímpica de Murillo</i> , <i>“uninorte”</i> encuentra <i>Universidad del Norte</i> .
Detalle de un lugar	Nombre, dirección, coordenadas, y la lista de rutas que pasan cerca (con la distancia al recorrido).

Para el asistente de IA — el diferenciador

El endpoint POST /assistant/ask recibe la pregunta del usuario en español natural (y opcionalmente su ubicación). Internamente Claude usa las mismas APIs que tú: busca lugares, encuentra rutas cercanas, calcula trayectos. La respuesta incluye **texto natural** + una **acción sugerida** opcional que tu UI puede convertir en un botón.

Ejemplo real probado

Usuario en Uninorte pregunta: “¿Cómo llego al Centro si voy de afán?”

Respuesta del backend:

“La opción más rápida es la ruta S12 (Sabanilla - Centro): esperas 44 segundos y llegas en 28 minutos aproximadamente. Si esa no viene pronto, tienes la C12 como alternativa (39 minutos totales). ¿Necesitas más info?”

Acción sugerida: abrir pin de espera en la ubicación del usuario.

Tipos de “acción sugerida” que puedes recibir:

type	Qué hace tu UI
START_TRIP	Mostrar botón “Iniciar viaje” con la ruta y destino sugeridos.
SHOW_ROUTE	Abrir el detalle de la ruta mencionada.
SHOW_LANDMARK	Abrir el detalle del lugar mencionado.
OPEN_WAIT_PIN	Sugerir crear pin de espera en la ubicación del usuario.
null	Solo respuesta textual, sin CTA.

Para cuenta de usuario y favoritos

Pregunta	Qué te da el backend
Crear cuenta (signup)	Email + password → recibes access_token + refresh_token + datos del usuario.
Iniciar sesión (login)	Igual que signup.

Refrescar sesión	Cuando el token expira (1 hora), úsalo para renovar sin pedir login.
Ver mi perfil	Nombre, email, ciudad, conteos de viajes y favoritos.
Ver mis favoritos	Lista de lugares + rutas guardados, con sus detalles embebidos.
Agregar favorito	Puede ser un lugar (landmark) o una ruta entera. Con alias opcional ("Casa", "Mi U").
Quitar favorito	Por ID.

Para viajes activos

Pregunta	Qué te da el backend
Iniciar un viaje	Recibe origen + destino + ruta. Calcula ETA. Solo puedes tener 1 viaje activo a la vez .
Mi viaje activo	Datos del viaje en curso o null si no hay.
Completar o cancelar	Marca el viaje. Emite evento WebSocket para que la UI se actualice en vivo.
Calificar viaje	1 a 5 estrellas + comentario opcional. Solo después de completarlo.

Para el “pin de espera” — feature potente para el pitch

El usuario marca un punto en el mapa donde piensa esperar el bus. El backend escanea cada 5 segundos si algún bus está acercándose. Cuando uno está a menos del tiempo configurado (por defecto 3 minutos), te **envía una notificación WebSocket** automáticamente.

Pregunta	Qué te da el backend
Crear pin de espera	Ubicación + ruta opcional + segundos antes de avisar. Te devuelve el ID.
Cancelar pin	Por ID.

Para reportar problemas

Pregunta	Qué te da el backend
Reportar incidente	Tipos: TRAFFIC (trancón), FULL_BUS (bus lleno), NO_BUS_PASSING (no pasa el bus), ACCIDENT.
Ver incidentes cerca de un punto	Lista de incidentes recientes en un radio (con el nombre del reportador).

Para la vista admin del pitch

Pregunta	Qué te da el backend
Snapshot de métricas	Usuarios activos, viajes en curso, preguntas a la IA por minuto, incidentes última hora, buses en servicio, pines de espera activos.
Feed de actividad	Lista unificada de TODO lo reciente: viajes, incidentes, preguntas a la IA, calificaciones, favoritos. Con filtro por fecha.

3 · Cómo conectarte al backend

URL base

En desarrollo (mientras corra el backend en mi máquina):

```
http://localhost:3000/api/v1
```

En producción (cuando esté desplegado):

```
https://[railway-url]/api/v1
```

Te paso la URL de Railway apenas hagamos deploy. Mientras tanto, Swagger interactivo en <http://localhost:3000/api/docs> es la fuente de verdad — puedes probar todos los endpoints desde el navegador.

Autenticación

Casi todos los endpoints requieren un JWT en el header. Lo obtienes al hacer login o signup. Si no lo mandas, recibes 401 Unauthorized.

```
Authorization: Bearer eyJhbGc...
```

Endpoints que NO requieren auth (los puedes llamar sin token):

- Mapa, búsqueda y discovery (lugares, rutas, buses-at-point)
- POST /auth/signup y POST /auth/login
- Healthcheck y métricas admin (por ahora públicos para facilitar la demo)

Tip de auth

El token expira a la hora. Guarda el refresh_token en localStorage o en una cookie y úsalo con POST /auth/refresh antes de que expire para no sacar al usuario.

4 · Mapping pantalla → endpoints

Las 5 pantallas core del MVP descritas en el brief, con los endpoints REST y los eventos WebSocket que cada una debería consumir.

Pantalla 1 · Mapa principal

Lo que ve el usuario: mapa de Barranquilla con buses moviéndose en vivo, lugares populares como anclajes, y bottom sheet con los próximos buses cerca.

Acción	Qué llamar
Al cargar el mapa	GET landmarks/nearby (anclajes visuales) + GET routes/:id/buses (snapshot inicial de buses)
Cuando usuario hace tap en el mapa	POST buses-at-point con esas coordenadas
Cuando usuario selecciona una ruta	GET routes/:id/corridor.geojson para dibujar el recorrido
WS — mientras el mapa esté abierto	bus_position (cada segundo, por cada bus) → actualizar posición de marcadores con animación fluida
WS — siempre	incident_reported → mostrar pin de incidente si está en el viewport

Pantalla 2 · Detalle de lugar

Cuando el usuario toca un anclaje (Uninorte, Buenavista, etc.), muestra el detalle.

Acción	Qué llamar
Al abrir	GET landmarks/:id
Para mostrar el próximo bus de cada ruta	POST buses-at-point con las coordenadas del lugar

Pantalla 3 · Asistente de IA

Input de texto donde el usuario pregunta libre. Respuesta + posible CTA.

Acción	Qué llamar
Al abrir	GET assistant/messages (historial)
Al enviar pregunta	POST assistant/ask con la pregunta + ubicación del usuario si la tienes
Al mostrar respuesta	Renderizar el campo answer + si hay suggested_action, renderizar como botón CTA

Importante

El endpoint del asistente tiene **rate limit de 5 preguntas por minuto por usuario** (Claude es caro). Si excedes, recibes 429 Too Many Requests. Muestra un mensaje amable y deshabilita el botón unos segundos.

Pantalla 4 · Viaje activo

Cuando el usuario inicia un viaje, esta pantalla muestra el bus moviéndose, tiempo restante, próximo paradero.

Acción	Qué llamar
Al iniciar viaje	POST trips con boarding_location + dropoff_location + route_id
Al cargar pantalla	GET trips/active (si no hubo POST inmediato)
Para dibujar el recorrido en el mapa	GET routes/:id/corridor.geojson
Al terminar el viaje	PATCH trips/:id con status COMPLETED
Al calificar	POST trips/:id/rating con stars (1-5) + comment opcional
WS — mientras el viaje esté activo	trip_update + bus_position en la sala trip:<id>

Pantalla 5 · Vista admin (la del pitch)

Mapa de Barranquilla con cientos de puntos animados + feed de actividad streaming + cards de métricas.

Acción	Qué llamar
Al cargar	GET admin/metrics (snapshot inicial) + GET admin/feed (últimos 50 eventos)
WS — suscribirse a la sala admin	Recibes TODOS los eventos (bus_position, trip_update, incident_reported, metrics_update, agent_action)
Cards de métricas	Animar el cambio cada vez que llegue metrics_update (cada 2s)
Feed streaming	Insertar nueva fila al tope cada vez que llegue agent_action o incident_reported o trip_update

5 · WebSocket — el tiempo real

Esto es **crítico**: el realtime no es opcional. Los buses se ven moverse en el mapa porque el backend los grita cada segundo por WebSocket. Sin WS la app parece estática y muerta.

Cómo conectarte

Cliente recomendado: `socket.io-client@4`.

```
import { io } from 'socket.io-client';

const socket = io(API_BASE, {
  path: '/realtime',
  transports: ['websocket', 'polling'],
});

socket.on('connect', () => {
  // Suscríbete a las salas que necesites
  socket.emit('subscribe', { room: 'city:BAQ' }); // todos los buses + incidentes
  socket.emit('subscribe', { room: 'trip:<tu-trip-id>' }); // tu viaje activo
  socket.emit('subscribe', { room: 'wait:<tu-wait-id>' }); // tu pin de espera
  // socket.emit('subscribe', { room: 'admin' }); // solo para la vista admin
});

socket.on('bus_position', (event) => {
  // event.busId, event.location, event.routeCode, ...
});
```

Las 6 salas (rooms) disponibles

Sala	Qué eventos recibes
admin	Todo lo que pasa en la ciudad. Solo úsala en la vista admin.
city:BAQ	Buses moviéndose + incidentes nuevos. Úsala en el mapa principal.
bus:<id>	Posición de un bus específico. Útil cuando trackeas un bus.
trip:<id>	Actualizaciones de tu viaje activo + posición del bus que tomaste.
wait:<id>	Alerta cuando el bus está cerca de tu pin de espera.
user:<id>	Notificaciones personales (reservado para post-MVP).

Los 6 eventos que vas a recibir

bus_position — cada segundo, por bus

Te llega la nueva posición GPS de un bus. Por cada bus visible en el viewport, dibuja un marcador.

```
{
  type: "bus_position",
  busId: "uuid",
  routeId: "uuid",
  routeCode: "C12",
  location: { lat: 11.012, lng: -74.812 },
  heading: 245,
  speedKmh: 28,
  fractionOfCorridor: 0.34,
  timestamp: "2026-05-23T14:30:00Z"
}
```

Tip clave para el mapa

No renderices saltos discretos. Cada segundo recibes una posición nueva. Para que el bus se vea moverse **suave**, anima entre la posición anterior y la nueva durante 1000 ms usando `framer-motion` o `requestAnimationFrame`. Sin esto los buses brincan y se ve mal en demo.

trip_update — cuando tu viaje cambia

Llega al iniciar trip, al completarlo, o cuando cambia el ETA / próximo paradero.

```
{
  type: "trip_update",
  tripId: "uuid",
  status: "IN_PROGRESS" | "COMPLETED" | "CANCELLED",
  remainingSeconds: 1240,
  currentLocation: { lat: ..., lng: ... }
}
```

wait_session_alert — tu bus está cerca

El usuario marcó pin de espera. Cuando el matcher detecta que el bus está a $\leq X$ segundos, te avisa.

```
{
  type: "wait_session_alert",
  waitSessionId: "uuid",
  busId: "uuid",
  routeCode: "C12",
  etaSeconds: 180,
  distanceM: 950
}
```

Sugerencia: dispara toast + vibración + sonido suave. Es el momento mágico del producto.

incident_reported — alguien reportó algo

Cuando un usuario reporta trancón, bus lleno, etc. Recibido por todos los suscriptos a la sala de la ciudad.

```
{
  type: "incident_reported",
  incidentId: "uuid",
  incidentType: "TRAFFIC" | "FULL_BUS" | "NO_BUS_PASSING" | "ACCIDENT",
  routeId: "uuid" | null,
  location: { lat: ..., lng: ... }
}
```

metrics_update — cada 2 segundos (solo admin)

Cards animadas de la vista admin: usuarios activos, viajes en curso, etc.

```
{
  type: "metrics_update",
  metrics: {
    activeUsers: 437,
    activeTrips: 89,
    aiQuestionsPerMinute: 23,
    incidentsLastHour: 4,
    busesInService: 142
  }
}
```

agent_action — solo admin, viene con el simulador

Cuando los 500 agentes virtuales (próximo bloque del backend) hagan algo: caminar, preguntar a la IA, iniciar viaje, etc. Esto alimenta el feed streaming del pitch.

```
{
  type: "agent_action",
  agentName: "María",
  agentProfile: "STUDENT_UNINORTE",
  action: "asked_ai" | "started_trip" | "boarded" | ...,
  payload: { ... },
  location: { lat: ..., lng: ... }
}
```

6 · Detalles importantes que no puedes olvidar

Convenciones de datos

Concepto	Formato
Coordenadas en REST y WS	Siempre {lat, lng} — nunca [lng, lat].
Coordenadas en GeoJSON	Solo el endpoint <code>corridor.geojson</code> usa [lng, lat] (es el estándar GeoJSON).
Distancias	Metros (números enteros).
Tiempos / ETAs	Segundos (números enteros) salvo cuando son fechas.
Fechas	ISO 8601 UTC: 2026-05-23T14:30:00Z

Shape uniforme de errores

Todos los errores del backend tienen este shape. Tu handler global puede confiar en él:

```
{
  "statusCode": 404,
  "message": "Trip not found",
  "error": "NotFound",
  "path": "/api/v1/trips/abc-123",
  "timestamp": "2026-05-23T14:30:00Z"
}
```

Mientras el backend no esté en producción

- Swagger interactivo en `http://localhost:3000/api/docs` (cuando corra el backend en mi máquina).
- Postman collection auto-generada desde Swagger — te la paso cuando deployemos.
- Mientras tanto, mockea con MSW usando este documento como contrato.
- Los eventos WS los puedes simular con un pequeño script Node emitiendo cada 1s — te paso el ejemplo si lo necesitas.

Lo único que falta del backend

Los 500 agentes virtuales (el simulador del pitch). Cuando esté listo, el evento `agent_action` empezará a llover en la sala `admin` y la vista del pitch va a verse increíble. **Todo lo demás ya está funcionando hoy** y lo puedes empezar a integrar.

Próximos pasos

- Te paso la URL pública de Railway apenas hagamos deploy.
- Mientras tanto, levanto el backend en mi máquina y te doy acceso si quieres probar contra mi DB.
- El simulador de 500 agentes lo tengo como último bloque del backend — debería estar listo antes del bloque de polish.
- Ping en Slack/WhatsApp para cualquier cosa que sea ambigua en este documento o en `docs/api-contract.md`.

— David Palacio · Backend lead Vialink